

Git & GitHub

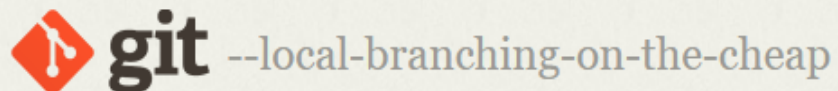
Tecnologías de Desarrollo de Software

Git & GitHub

- Git: Sistema de control de versiones
 - Distribuido
- GitHub: Plataforma para desarrollo colaborativo
 - Basado en Git

Git

- Git es un sistema de control de versiones distribuido (DCVS)
 - Repositorio remoto
 - Repositorios locales.
- Creado por Linus Torvalds durante el desarrollo de Linux (open source project) con los siguientes objetivos:
 - Diseño simple, fácil de usar y rápido
 - Buen soporte de desarrollo no lineal (manejo de miles de ramas)
 - Soporte de grandes proyectos
 - Completamente distribuido (una copia de la base de datos del proyecto en los ordenadores de cada miembro del equipo de trabajo)
- Características: (<https://git-scm.com/book/en/v2/Getting-Started-What-is-Git%3F>):
 - Se puede trabajar sin conexión (repositorios locales).
 - Integridad de datos



About

Documentation

[Reference](#)[Book](#)[Videos](#)[External Links](#)

Downloads

Community

This book is available in [English](#).

Full translation available in

[български език](#),

Book

The entire Pro Git book, written by Scott Chacon and Ben Straub and published by Apress, is available here. All content is licensed under the [Creative Commons Attribution Non Commercial Share Alike 3.0 license](#). Print versions of the book are available on [Amazon.com](#).



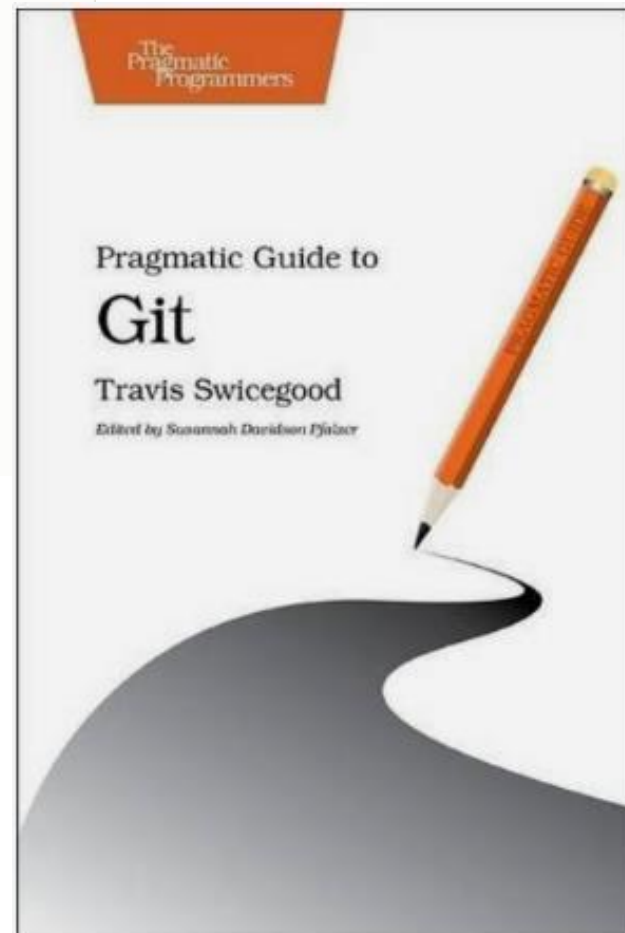
2nd Edition (2014)

[Download Ebook](#)



1. Inicio - Sobre el Control de Versiones

- 1.1 [Acerca del Control de Versiones](#)
- 1.2 [Una breve historia de Git](#)
- 1.3 [Fundamentos de Git](#)
- 1.4 [La Línea de Comandos](#)



GitHub

Crear un nuevo repositorio

Privado. Solo visible para el grupo

Si se marca, el repositorio remoto tendrá un primer commit.
No marcar si ya se han creado otros commits en el repositorio local

Opcional. Preferible hacerlo
Manualmente en el primer commit.

Create a new repository

A repository contains all project files, including the revision history. Already have a project repository elsewhere? [Import a repository](#).

Required fields are marked with an asterisk (*).

Repository template

No template ▾

Start your repository with a template repository's contents.

Owner *

jesusc ▾

Repository name *

pds-2025

✓ pds-2025 is available

Great repository names are short and memorable. Need inspiration? How about [jubilant-telegram](#) ?

Description (optional)

Prácticas de PDS para el curso 2025

- ☐  **Public**
Anyone on the internet can see this repository. You choose who can commit.
- ☒  **Private**
You choose who can see and commit to this repository.

Initialize this repository with:

- ☐ Add a README file
This is where you can write a long description for your project. [Learn more about READMEs](#).

Add .gitignore

.gitignore template: None ▾

Choose which files not to track from a list of templates. [Learn more about ignoring files](#).

Choose a license

License: None ▾

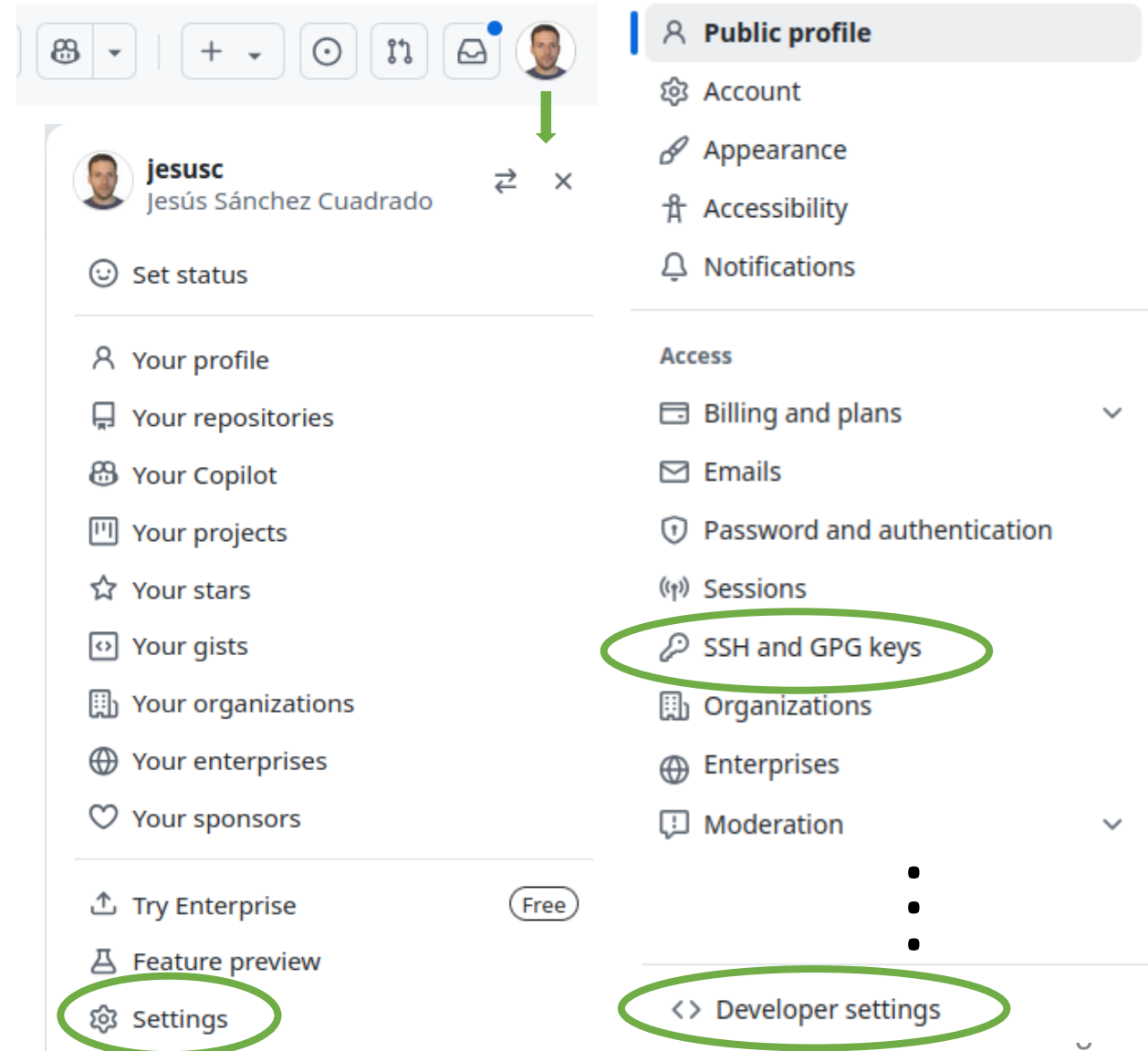
A license tells others what they can and can't do with your code. [Learn more about licenses](#).

ⓘ You are creating a private repository in your personal account.

Create repository

Configurar acceso

- Acceso a repositorios remoto
 - GitHub ya no soporta acceso a los repositorios (git pull/push) utilizando usuario y contraseña
- Dos formas:
 - SSH
 - API Key (se usa como password)
- Perfil -> Settings
 - SSH and GPG keys
 - Developer settings
 - API Token (classic)



Configurar acceso

- Acceso por SSH
- Hay que generar el par clave privada – pública

```
ssh-keygen -o -t rsa -C "tu@email.com"  
# La clave pública está en /home/tu/.ssh/id_rsa.pub.  
# Hay que copiarla en GitHub, `Account settings -> SSH Keys`
```

- Se generan dos ficheros:
 - `id_rsa` : clave privada, ¡no compartir!
 - `id_rsa.pub` : clave pública, copiar contenido en GitHub

```
ssh-rsa  
BBBBB3NzaC1yc2EAAAADAQABAAQDTm+fHGazjj0T+aY1  
xsZGgu83234aGdfGskqCuzUSW/op9RpRmLVtohpX
```



Add new SSH Key

Title

Key type

Authentication Key ↕

Key

Begins with 'ssh-rsa', 'ecdsa-sha2-nistp256', 'ecdsa-sha2-nistp384', 'ecdsa-sha2-nistp521', 'ssh-ed25519', 'sk-ecdsa-sha2-nistp256@openssh.com', or 'sk-ssh-ed25519@openssh.com'

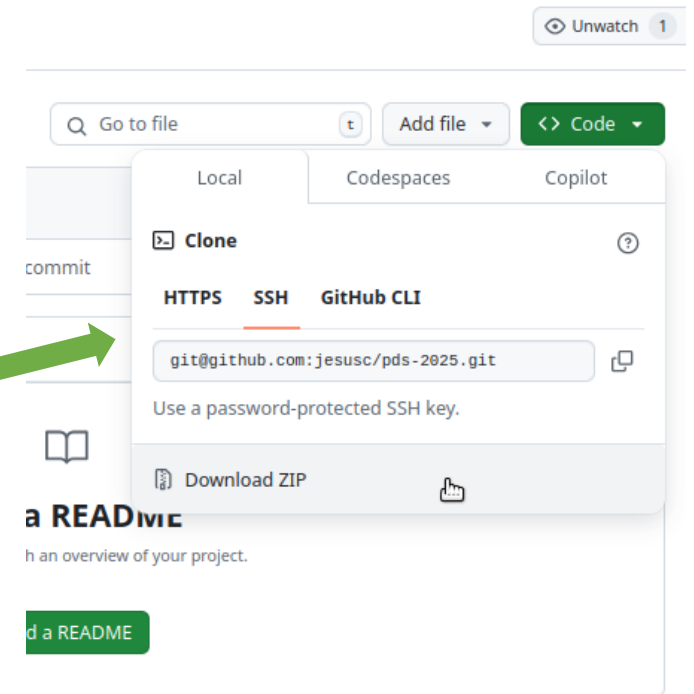
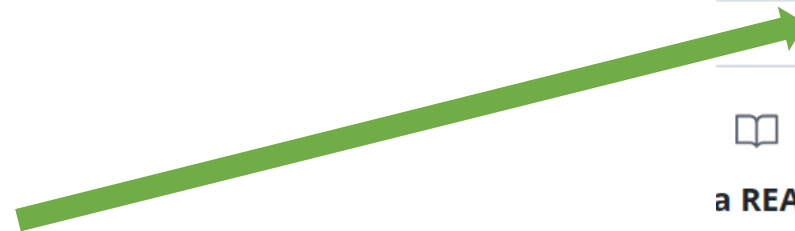
Add SSH key

Git

- Dos formas de empezar a usar un repositorio
 - Crear un repositorio local vacío + crear repositorio en GitHub
 - Clonar un repositorio de GitHub
 - Lo más sencillo: crear el repositorio en GitHub y luego clonarlo

- Inicializar un repositorio
`$ git init`

- Clonar un repositorio
`$ git clone URL`



Git

- Configuración inicial

- Especificar los datos personales para que se pueda registrar quién ha hecho un *commit*.

```
git config --global user.name "Tu Nombre"  
git config --global user.email "tuemail@ejemplo.com"
```

- En el ordenador del laboratorio, no usar `--global`, para que solo se aplique al repositorio actual.

Git - .gitignore

- Indica a git qué archivos debe ignorar (no añadirlos al repositorio)
 - No incluir ficheros binarios
 - No incluir ficheros generados
 - No incluir configuraciones locales
- Usar <https://gitignore.io>
 - Generar fichero .gitignore para las tecnologías de tu proyecto

```
# Compiled class files
*.class

# Packages files
*.jar
*.war

# Excluir de las reglas
!lib/especial.jar
```

gitignore.io

Create useful .gitignore files for your project

Java X

Maven X

Create

Git – Uso rápido

- **git init**
 - Inicializar un repositorio
- **git add <fichero>**
 - Añadir un fichero al área de “staging”
 - Decirle a git que cuando hagamos un “commit” queremos que se guarden los cambios de ese fichero
- **git commit**
 - Guardar los cambios
 - Ojo! Solo aquellos que se han planificado con “git add”
- **git push**
 - Guardar cambios en un repositorio remoto
 - ¿Cómo configuramos ese repositorio remoto? Típicamente GitHub

Ejercicio rápido

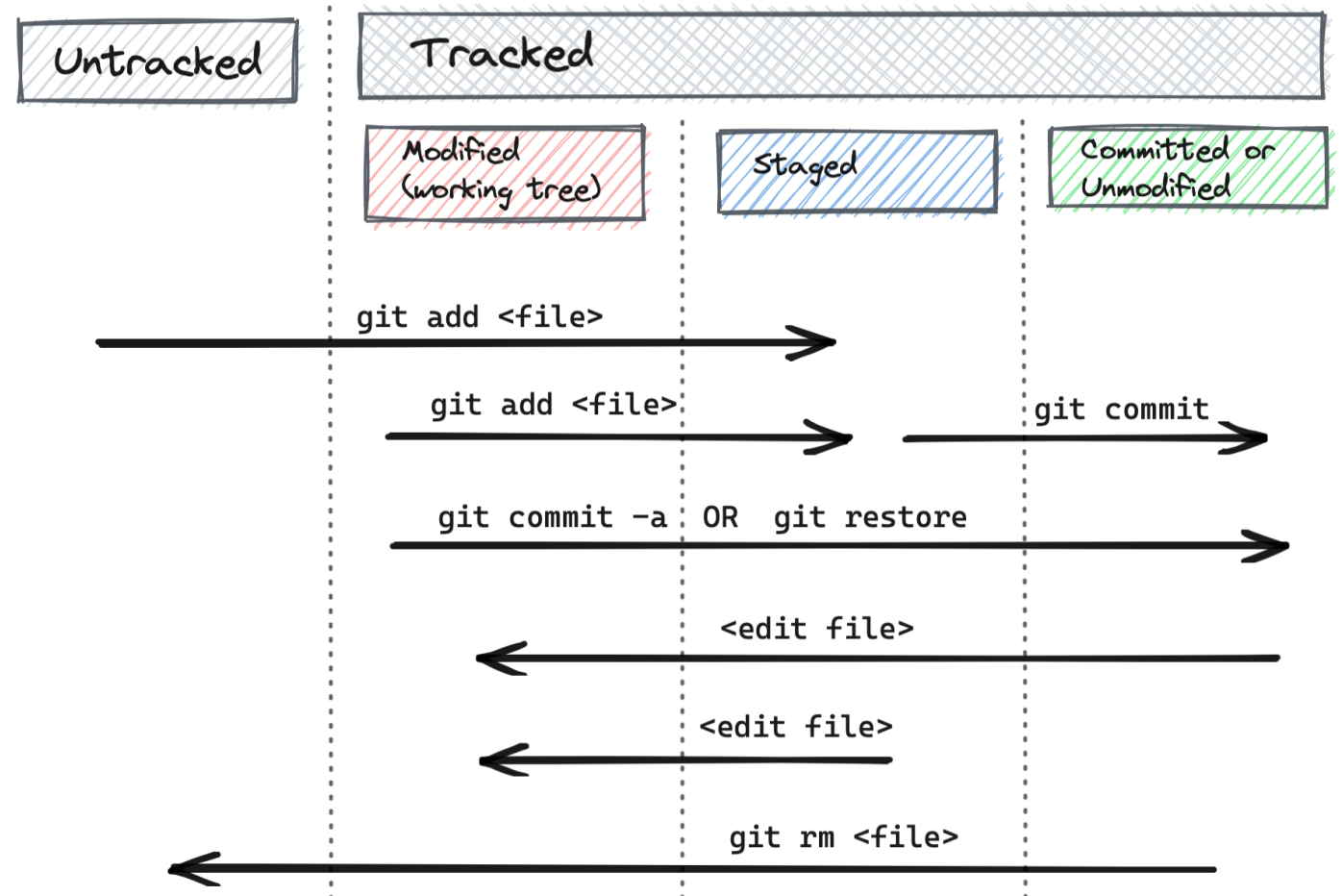
- Crear un repositorio en GitHub: prueba-tds
- Clonar el repositorio
 - `git clone https://github.com/<usuario>/prueba-tds`
- Editar el fichero README.md
- Añadirlo para el siguiente commit
 - `git add README.md`
- Hacer un commit para indicar a git que los cambios son firmes
 - `git commit` ó `git commit -m "Primer cambio"`
- Subir los cambios al repositorio remoto (GitHub)
 - `git push origin main`
- Ver que en GitHub el commit está guardado

Git – Áreas de un repositorio

- Working Directory
 - Donde editamos los ficheros.
- Staging Area (Index)
 - Un área temporal donde se guardan los cambios antes de hacer *commit* de ellos
- Repositorio local
 - Contiene el código que se ha guardado en firme (*committed*)
- Repositorio remoto
 - Versión del repositorio en otra localización (típicamente un servidor)

Git - Estados de un fichero

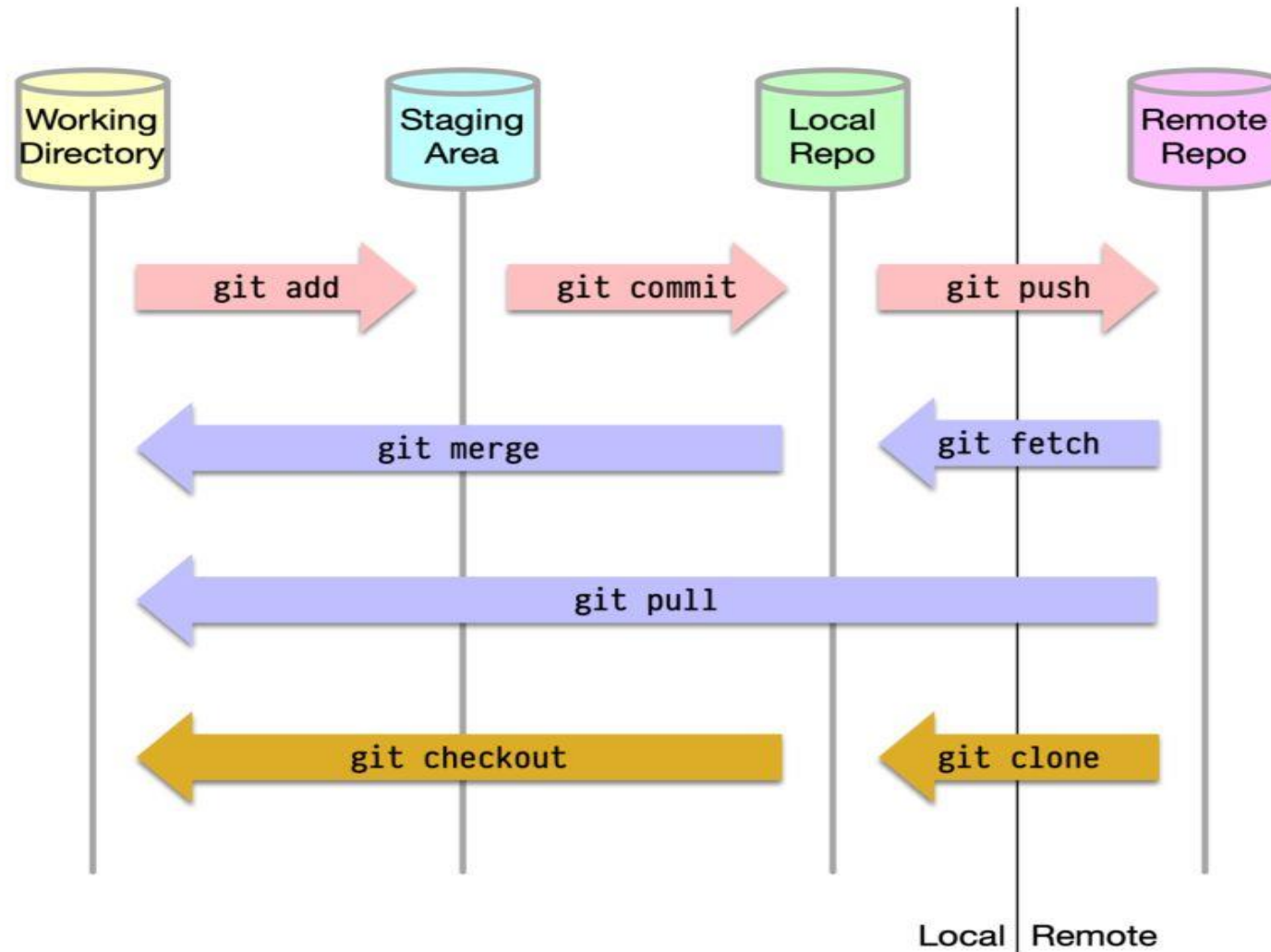
- Untracked
- Modified
- Staged
- Committed



Git – Áreas y comandos

How Git Commands work

ByteByteGo.com



* https://www.youtube.com/watch?v=e9lnsKot_SQ

Conceptos: “main”, “origin” y “HEAD”

- **main**: rama predeterminada que se crea automáticamente cuando se crea un repositorio.
 - *main* se considera la rama principal y la raíz de la mayoría de las demás ramas.
 - Se encuentra el "código definitivo", que luego va a producción, y es **la rama en la que se mezclan todas las demás tarde o temprano** para dar por finalizada una tarea e incorporarla al producto final

Conceptos: “main”, “origin” y “HEAD”

- *origin*: el nombre predeterminado que recibe el repositorio remoto principal contra el que trabajamos.
 - Para nombrar una rama del repositorio remoto, ej., “origin/main”
- *HEAD*: se refiere al *commit* en el que está tu repositorio posicionado en cada momento.
 - Por regla general *HEAD* suele coincidir con el último *commit* de la rama en la que se está, ya que habitualmente se trabaja en lo último.
 - Pero si se mueve hacia otro *commit* anterior, entonces el *HEAD* estará más atrás.

Flujo de trabajo básico

- Añadir archivos al área de staging
 - Cada vez que se hace un cambio a un archivo, pasa al estado “cambios no rastreado” (*untracked*).

```
git add <archivo>    # Agrega un solo archivo
git add .             # Agrega todos los archivos
```

- Hacer un commit
 - Guarda los cambios que están añadidos al área de *staging*

```
git commit -m "Mensaje descriptivo del cambio"
```

Flujo de trabajo básico

- Comprobar el estado del repositorio

```
$ git status
```

Staged changes

Modificaciones añadidas al área de staging con `git add`.

En la rama main
Tu rama está adelantada a 'origin/main' por 1 commit.
(usa "git push" para publicar tus commits locales)

Cambios a ser confirmados:

(usa "git restore --staged <archivo>..." para sacar del área de stage)

modificados: .gitignore

Untracked changes

Modificaciones no añadidas al área de *staging*.

Cambios no rastreados para el commit:

(usa "git add <archivo>..." para actualizar lo que será confirmado)

(usa "git restore <archivo>..." para descartar los cambios en el directorio de trabajo)

modificados: README.md

Untracked files

Ficheros que todavía no han sido añadidos al repositorio (pero están en el *working directory*)

Archivos sin seguimiento:

(usa "git add <archivo>..." para incluirlo a lo que se será confirmado)

app/
docs/

Flujo de trabajo básico

- Historial de cambios

\$ git log

Comentario del commit

ID del commit

Ramas que apuntan al commit

```
commit 048f291d57ae3c80f0bc4ed5a1369977b9865409 (HEAD -> main)
Author: Jesús Sánchez Cuadrado <jesusc@um.es>
Date: Thu Jan 23 00:21:48 2025 +0100

    Nuevo README.md

commit 060807e5b0f5a62355f2f4d8b225703357b6543c
Author: Jesús Sánchez Cuadrado <jesusc@um.es>
Date: Wed Jan 22 23:53:43 2025 +0100

    Initial README.md

commit 7f8e506241c90bfae28847e6b73a924048fe93dd (origin/main, origin/HEAD)
Author: Jesús Sánchez Cuadrado <jesus.sanchez.cuadrado@gmail.com>
Date: Wed Jan 22 16:52:22 2025 +0100

    Initial commit
```

Flujo de trabajo básico

- Actualizar el repositorio local desde el remoto

```
$ git fetch
```

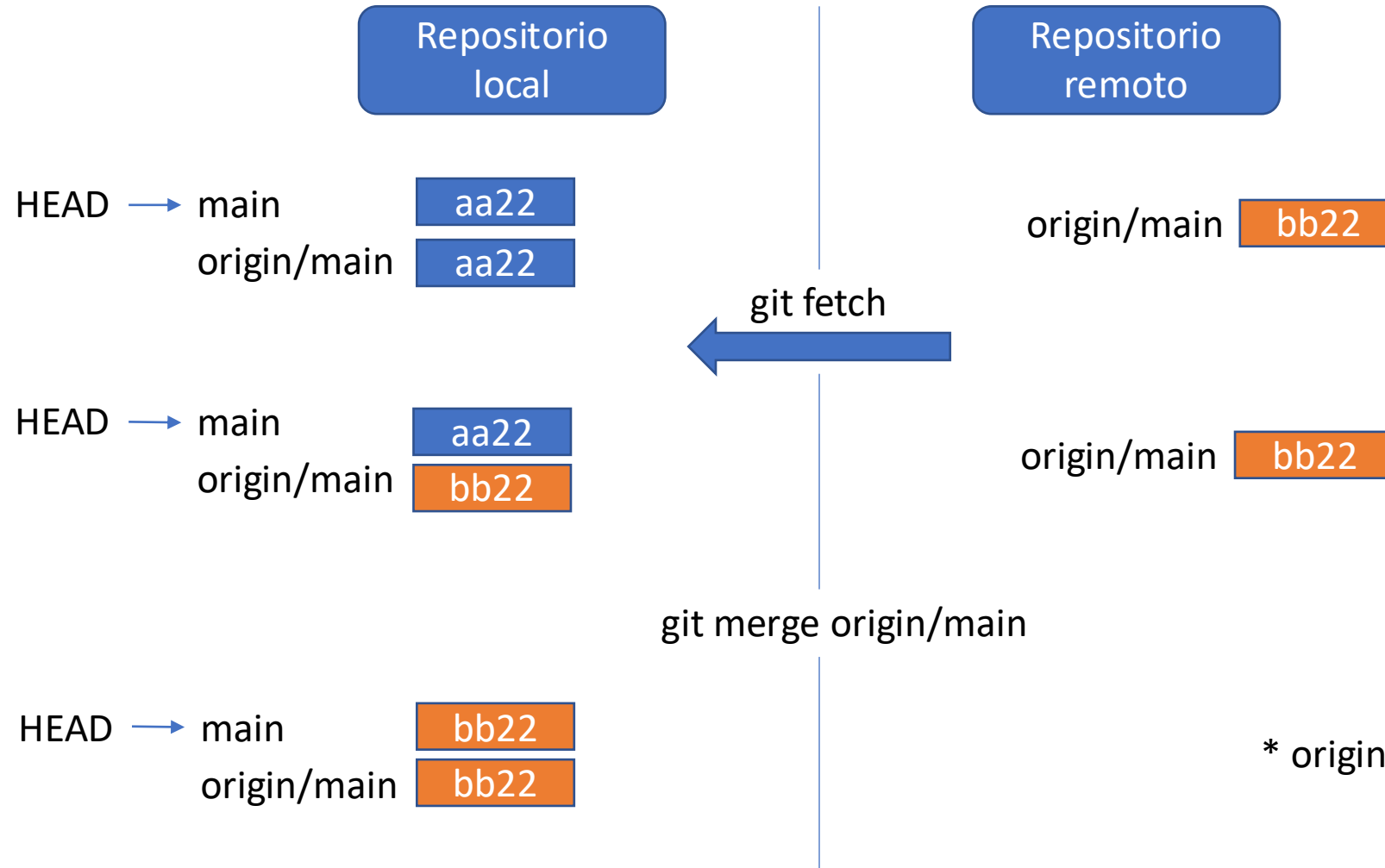
```
Desde github.com:jesusc/pds-2025  
* [nueva rama]      jesus-prueba -> origin/jesus-prueba
```

← Lista de ramas que se han
descargado como ramas remotas

```
$ git merge origin/<mi-rama> ← La rama remota de interés
```

Git – Áreas y comandos

- ¿Qué sucede cuando hacemos `git fetch`?



* `origin/main` es de solo lectura

Trabajar con ramas

- Crear una rama

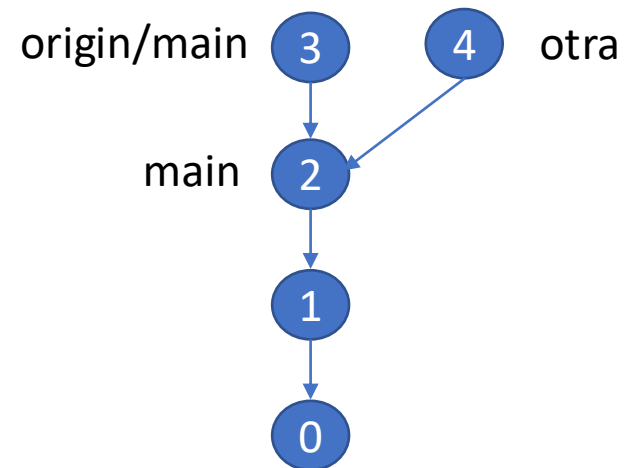
```
$ git branch <rama>           # Crear rama  
$ git checkout -b <rama>       # Crear y cambiar de rama
```

- Cambiar de rama

```
$ git switch <rama>           # También, git checkout <rama>
```

Trabajar con ramas

- ¿Qué es una rama?
 - Un puntero a un commit
 - ¿Por qué funciona esto?
 - Porque cada *commit* tiene un puntero a su padre (al *commit* anterior)



Trabajar con ramas

- Mezclar ramas
 - Ojo, una rama local y una rama remota son ramas que se integran de manera normal con un *merge* (ver git pull)
 - El comando *git merge* mezcla la rama especificada con la rama actual

```
$ git merge <otra-rama>
```

Flujo de trabajo básico

- Conflictos durante un merge

```
Auto-merging <archivo>
```

```
CONFLICT (content): Merge conflict in <archivo>
```

- Pasos para realizar un merge

- Revisar los conflictos

- Los archivos con conflictos tendrán secciones delimitadas por <<<<<, ===== y >>>>>.

- Editar y resolver los conflictos

- Selecciona los cambios que deseas conservar inspeccionando con el editor

- Marcar como resuelto

```
git add <archivo>
```

```
git commit -m "Resolver conflictos"
```

- El commit es un *merge commit*

Flujo de trabajo básico – Resolver conflictos



Hola mundo
Adiós mundo

fichero.txt

git add fichero.txt
git commit
git push

git checkout -b rama1

Hola mundo
desde rama1
Adiós mundo

fichero.txt

git commit -a -m "cambios rama1"

git fetch → descarga origin/rama2

git merge origin/rama2 → ¿por qué no rama2 solo?



git fetch
git checkout main

git checkout -b rama2

Hola mundo
desde rama2
Adiós mundo

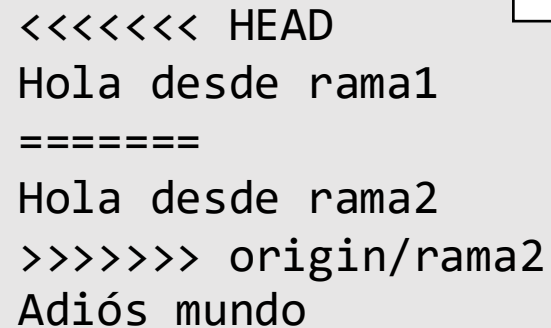
fichero.txt

git commit -a -m "cambios rama2"
git push origin rama2

Flujo de trabajo básico – Resolver conflictos

Cambio en la rama a la
que apunta HEAD (actual)

Cambio “entrante”



```
<<<<<< HEAD
Hola desde rama1
=====
>>>>>> origin/rama2
Hola desde rama2
Adiós mundo
```

fichero.txt

- Resolver el conflicto
 - ¿Qué tres cosas podemos hacer?
 - Quedarnos con el cambio de la rama actual
 - Quedarnos con el cambio que viene de la rama que estamos mezclando
 - Unir ambos cambios (ej., “Hola desde rama1 y rama2”)
 - Hay que borrar las marcas
 - Se puede usar una herramienta de merge
 - `git mergetool` abre una (ej., meld)

Flujo de trabajo básico – Trabajo sobre la rama main



Hola mundo
Adiós mundo

fichero.txt

git add fichero.txt
git commit
git push

Hola mundo
desde rama1
Adiós mundo

fichero.txt

git commit -a -m "cambios rama1"

git fetch → descarga origin/main

git merge origin/main



git fetch
git checkout main

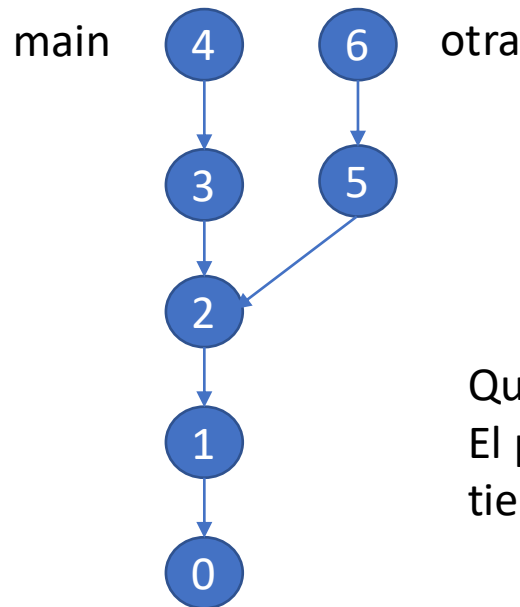
Hola mundo
desde rama2
Adiós mundo

fichero.txt

git commit -a -m "cambios rama2"
git push origin main

Mezcla de ramas

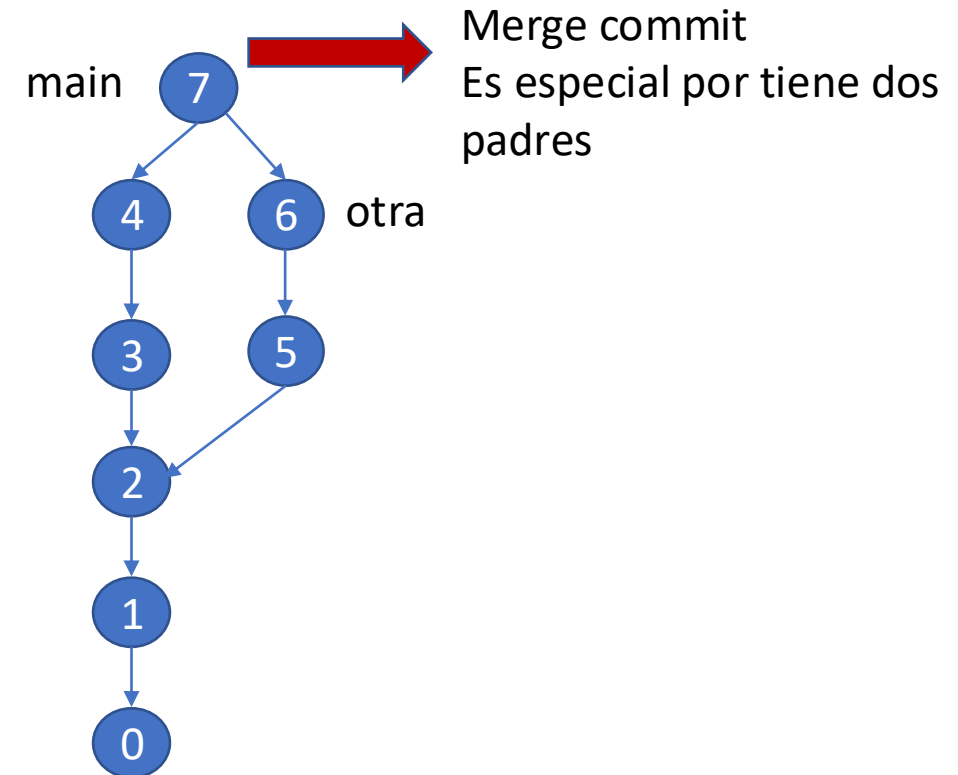
- Cuando se mezclan dos ramas, si las ramas han divergido, se crea un "merge commit"
 - Dos ramas han divergido si "una no es continuación de la otra"



git merge otra



Queremos integrar "otra" en "main"
El problema es que el commit "5"
tiene como padre "2"



Flujo de trabajo básico

- Sincronizar cambios locales con el repositorio remoto

```
$ git push origin <rama>
```

- Problema típico

- Rama desactualizada respecto a la rama remota

- La rama remota contiene cambios que no están en el local

```
error: failed to push some refs to '<URL>'
```

```
hint: Updates were rejected because the remote contains work that you do not have locally.
```

- Solución: Obtener los cambios remotos (*fetch*) y sincronizar con los locales (*merge*)

Otros comandos útiles

- Apartar cambios

```
$ git stash
```

```
# Después, para recuperar los cambios
```

```
$ git stash pop
```

- Útil para solucionar este error al hacer un *pull*

```
error: Your local changes to the following files would be overwritten by merge: <archivo>  
Please commit your changes or stash them before you merge.
```


GitHub

- Elementos básicos:

- Issues

- Seguimiento de las tareas. Se puede usar para:

- Por cada tarea, un issue
 - Por cada funcionalidad, un issue
 - Por cada bug, un issue
 - Permiten discutir y colaborar

- Pull requests

- Dada una rama, un *pull request* representa una propuesta de cambio sobre la rama principal.
 - Permiten revisar, comentar y aprobar modificaciones antes de integrarlas en la rama principal,

- Markdown para escribir documentación (ficheros .md), issues y pull requests

- Formato para escribir documentación.
 - <https://docs.github.com/es/get-started/writing-on-github/getting-started-with-writing-and-formatting-on-github/basic-writing-and-formatting-syntax>

Demostración

- Repositorio de ejemplo:
 - <https://github.com/jesusc-umu/ejemplo-pds-2025>

GitHub – Issues

- <https://github.com/jesusc-umu/ejemplo-pds-2025/issues/1>
- Descripción
 - Indicará qué representa la tarea, qué se pretende hacer
- Imágenes
 - Mermaid
 - Subir imágenes al issue

Inscripción de jugadores #1



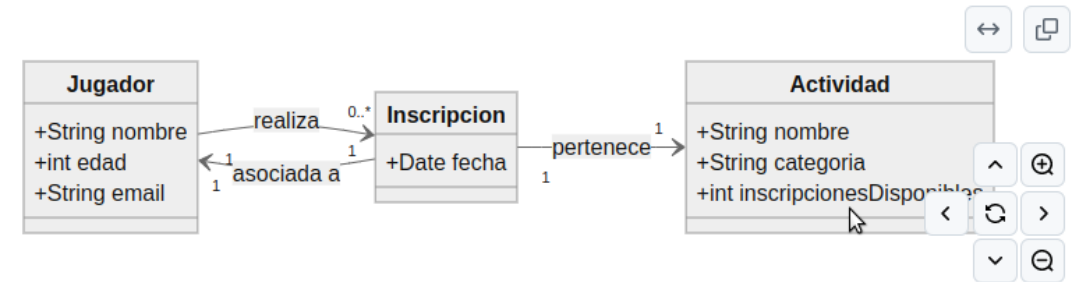
jesusc opened 20 hours ago · edited by jesusc-umu

Edits ...

Descripción

Funcionalidad básica para inscribir jugadores.

Modelo de dominio

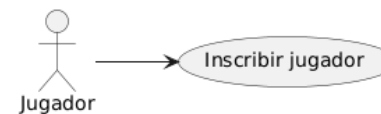


Caso de uso

Nombre: Inscribir jugador en deporte individual

Diagrama

```
@startuml
left to right direction
:Jugador: --> (Inscribir jugador)
@enduml
```



GitHub – Pull requests

Estructura del proyecto Maven/Java #2

Edit <> Code

Open jesusc wants to merge 1 commit into `main` from `proyecto-maven`

Conversation 0 Commits 1 Checks 0 Files changed 4

+110 -1



jesusc commented now

Collaborator

No description provided.



Estructura del proyecto Maven/Java

03434ab



This branch has no conflicts with the base branch
Merging can be performed automatically.

Merge pull request or view [command line instructions](#).

Reviewers

No reviews

Still in progress? [Convert to draft](#)

Assignees

No one—[assign yourself](#)

Labels

None yet

Projects

None yet

Relaciones entre issues y pull request

- Cada issue o PR tiene un número asignado
- Referenciado como ej., #37
- Modificadores especiales en PR
 - Fixes #37
 - Significa que cuando el PR se integre en la rama principal, el *issue* se cerrará automáticamente